

PLANIFICACION DE PROYECTO

PEAKFIT

CLI en Node.js para gestion de clientes, planes de entrenamiento, nutricion y finanzas

[Nombre del estudiante] | [Curso] | [Fecha]

Planeacion y documentacion

Como entendi el problema antes de construir la solucion

1 Leer el enunciado

Identifique las 6 funcionalidades pedidas: clientes, planes, seguimiento, nutricion, contratos y finanzas.

2 Modelar los datos primero

Disene el esquema SQL antes de escribir Javascript: entidades, relaciones y reglas (CHECK, FK) definidas desde el inicio.

3 Documentar el flujo

Escribi en el README que hace cada menu antes de programarlo, para no improvisar la logica de negocio.

Entidades identificadas en la planeacion

Cliente	Plan de entrenamiento	Contrato
Seguimiento fisico	Plan de alimentacion	Movimiento financiero

Documentacion base: **README.md** con el flujo de cada menu, y el esquema **peakfitdb.sql** con las tablas, llaves foraneas y reglas de validacion (CHECK) definidas antes de escribir un solo modelo en Javascript.

Preguntas que guiaron la planeacion

- Que pasa si un cliente cancela un plan a medio camino? (definio la necesidad de rollback)
- El contrato se crea a mano o se genera solo al asignar el plan? (definio usar una Factory)
- Que operaciones no pueden quedar a medias, como un pago sin contrato? (definio las transacciones)

Eleccion de patrones de diseno

Por que Repository y Factory, y que problema resuelve cada uno

1

PATRON REPOSITORY

PROBLEMA Sin una capa fija, el SQL quedaria repetido y mezclado dentro de la logica de negocio, dificil de mantener o de cambiar de base de datos.

SOLUCION Una sola capa (repositories/) concentra todas las consultas. Los servicios piden datos por intencion ("buscarPorId") sin saber como se guardan.

Carpeta: scr/repositories/

2

PATRON FACTORY

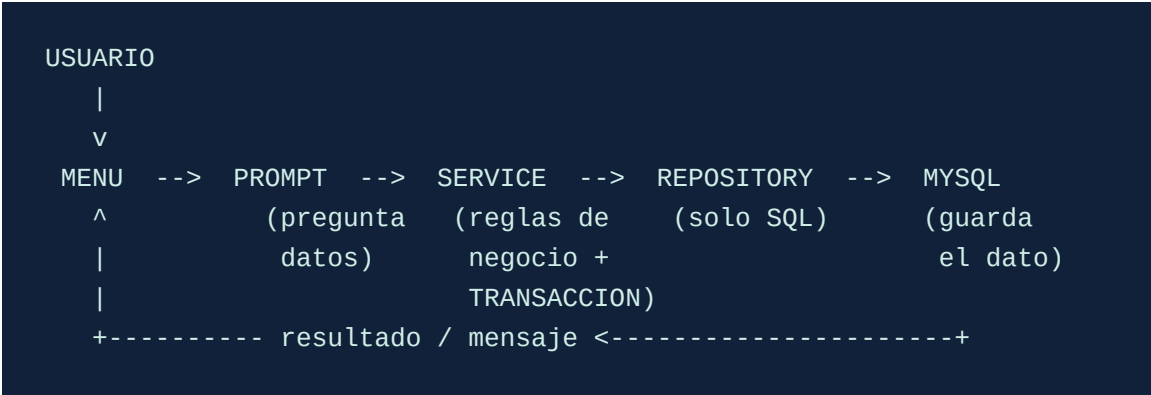
PROBLEMA El contrato debe generarse automaticamente al asignar un plan, siempre con las mismas reglas: fecha fin, precio y condiciones.

SOLUCION Una clase (ContratoFactory) centraliza esa construccion. Nadie mas calcula fechas o precios; si cambian las reglas, se cambian en un solo lugar.

Carpeta: scr/factories/

Flujo de ejecucion pensado

El camino que sigue un dato, planeado de antemano en capas



Decision clave: acciones criticas usan transaccion

Antes de programar, definimos que acciones no pueden quedar a medias. Se agruparon en un "todo o nada": si un paso falla, se deshacen los anteriores (rollback).

Accion	Que se hace junto
Asignar plan	Crear contrato + registrar el pago
Renovar plan	Cerrar contrato viejo + crear uno nuevo + pago
Cancelar plan	Borrar seguimiento fisico + marcar contrato cancelado
Registrar avance	Guardar el registro + sus fotos